АГЕНТЫ ИИ / УПРАВЛЕНИЕ API / ПРОТОКОЛ КОНТЕКСТА МОДЕЛИ (MODEL CONTEXT PROTOCOL, MCP)

Разработка API для агентов

Узнайте, как компания Webflow переработала свои API для агентов ИИ, перейдя от обертывания конечных точек к протоколу контекста модели, ориентированному на интенты.

1 августа 2026 г. в 10:00 Янь Се, Вират Патель и Альберт Чанг

Каролина Грабовска для Unsplash+

Компания Webflow спонсировала этот пост.

В начале 2025 года компания Webflow начала разработку для MCP до того, как появился чёткий план действий по созданию API, готовых к использованию агентами. Мы публично анонсировали наш MCP-сервер в апреле 2025 года, а затем присоединились к демонстрационному дню Cloudflare в мае 2025 года вместе с первой волной удалённых MCP-серверов.

подходят для агентов. Разработчики API исходят из того, что люди могут читать документацию, проводить исследования или получать ответы из различных внешних источников, создавать детализированные конечные точки, управлять состоянием и обрабатывать сбои вручную. Агенты работают в гораздо менее очевидном контексте и в значительной степени полагаются на сам API, чтобы планировать и выполнять задачи, а также решать, стоит ли повторить попытку, внести изменения или остановиться. Таким образом, прямое предоставление API-интерфейсов для разработчиков агентам делало наши первые инструменты MCP слишком низкоуровневыми и «болтливыми». Для выполнения простых задач часто требовалось много вызовов инструментов, а с более сложными рабочими процессами возникали проблемы. Это приводило к незаметным сбоям, неэффективному времени выполнения и ненадежному выполнению задач.

«API, разработанные для разработчиков, плохо подходят для агентов. Разработчики API исходят из того, что люди могут читать документацию, проводить исследования, создавать детализированные конечные точки, управлять состоянием и устранять сбои вручную».

В течение следующего года мы провели масштабную работу по доработке: переработали инструменты, ориентируясь на интенты, а не на конечные точки, упростили схемы, повысили эффективность вызова инструментов и сделали так, чтобы агентам было проще анализировать ответы. Мы также продолжаем вкладывать значительные средства в базовые компоненты — от уровня кода до абстракций файловой системы — для поддержки более надежных и функциональных рабочих процессов с участием агентов.

Многие из этих решений теперь соответствуют набирающим популярность отраслевым шаблонам: инструменты, ориентированные на выполнение задач, понятные агентам схемы, практические рекомендации по устранению ошибок и API, разработанные с учетом интентов, а не особенностей реализации. В этом посте мы расскажем об уроках, которые легли в основу сервера MCP от Webflow, и о том, почему разработка для агентов в конечном ·ге стала гораздо более масштабной задачей, чем предоставление API ·ез MCP. Это потребовало переосмысления того, как автономные системы

выполнения.

Почему API для разработчиков плохо подходят для агентов

Пользователь может попросить агента обновить главную страницу. В традиционном API это может превратиться в цепочку низкоуровневых операций:


```
list pages
→ find homepage
→ fetch page content
  · inspect content tree
  → find hero section
```

tools:

```
getPages()
getPageContent(pageId)
updateContent(contentId, payload)
publishPage(pageId)
```

MCP itself was not the problem. The protocol worked as intended. An LLM only sees tool descriptions, schemas, previous responses, and the current context window. It has to infer execution strategy dynamically while preserving intermediate state across multiple tool calls. That's where the failure modes begin to appear. Every additional tool call increases latency, token consumption, context pressure, and the probability of execution failure. The agent now has to:

TRENDING STORIES

1. Designing APIs for agents
2. The MCP debate has a context problem
3. APIs aren't dead. Here's where MCP fits alongside them.
4. 10 strategies to reduce MCP token bloat
5. Why the Model Context Protocol Won

- preserve intermediate identifiers correctly
- select the right resource from ambiguous results
- interpret noisy or overly generic responses
- coordinate dependent operations in sequence
- decide whether failures are retryable or terminal

Even when the API technically supports the workflow, too much of the model's runtime budget goes into navigating the API instead of completing the user's task.

What an agent API needs

lesson for us was that exposing APIs through MCP was only the starting point. ...liable agent execution depended much more on the shape and semantics of the

ambiguous choices, fewer dependent calls, less state to preserve, and clearer guidance when something goes wrong.

“Developer APIs typically optimize for flexibility and composability. Agent APIs need to optimize for execution reliability: fewer ambiguous choices, fewer dependent calls, less state to preserve, and clearer guidance when something goes wrong.”

For the homepage hero update, the better interface is a task-level tool that matches the user’s intent directly:

```
update_page_section({
  page: "home",
  section: "hero",
  changes: {
    heading: "Build faster with Webflow"
  }
})
```

Declarative APIs turn complex, multi-step workflows into a single intent-driven call. Rather than requiring the agent to discover internal structures, preserve intermediate state, coordinate dependent operations, or recover from partial failures, the platform handles orchestration internally. This reduces failure modes and lets the agent focus on the user’s goal instead of execution details.

Moving from low-level APIs to task-level tools was directionally correct, but it introduced a new problem at Webflow’s scale: tool explosion. Webflow supports too many valid workflows to model each one as its own MCP tool. A purely task-oriented design quickly creates a large, overlapping tool surface that becomes difficult for agents to navigate efficiently. To solve this problem, there are basically 2 approaches, and we’ll discuss both patterns in more depth in the sections that follow:

1. **A layered tool architecture:** Instead of exposing hundreds of standalone tools, we grouped related capabilities into broader domain-level tools with typed composable actions inside them. This gave agents a more structured and navigable map of the product surface while keeping the top-level tool space manageable.

interaction model from API orchestration toward execution environments and code-driven manipulation.

Launching and scaling our MCP server

One of the lessons we learned was that designing APIs for agents extends beyond the API surface itself. Once agents became active participants in long-running workflows, many surrounding system decisions started to directly affect execution quality: session architecture, tool organization, runtime coordination, observability, and even distribution.

In practice, building reliable agent systems became a full-stack design problem. This shaped how we approached the Webflow MCP server across four areas:

- infrastructure for stateful execution and runtime coordination
- tool surface design for clarity and efficient capability discovery
- observability systems for understanding real agent behavior
- distribution patterns that reduced setup friction and scaled adoption

1. Infrastructure

The infrastructure requirements for an MCP server turned out to be very different from a traditional API service. While MCP itself does not require server-side state, real-world agent workflows are highly stateful. An agent may authenticate once, inspect site context, call multiple tools across different product surfaces, and continue reasoning through a long-running workflow. Treating every interaction as fully stateless would force the model to repeatedly reconstruct context and execution state, increasing both latency and failure probability.

We built the Webflow MCP server on Cloudflare using Durable Objects because they provided a natural session-oriented execution model. Each MCP session owns its own server instance, managing user context, available tools, and runtime coordination without requiring separate session infrastructure.

We also had to support two different execution environments. Some operations run headlessly through Webflow APIs, while Designer actions — like inserting elements or updating styles — depend on the live browser canvas. For those workflows, the MCP server maintains a WebSocket bridge to a Designer Extension running inside the user's active Webflow session.

been progressively moving more Designer capabilities into headless APIs. Reducing dependence on live browser execution simplifies the runtime architecture, lowers coordination complexity, and improves overall stability and reliability for agent workflows.

2. Designing the tool surface

As mentioned earlier, we found two different approaches that help reduce orchestration and discovery overhead for agents: layered tool organization and filesystem abstractions with code representations of Webflow projects. Over time, we started viewing them as short-term and long-term solutions to the same underlying problem.

In the short term, the layered tool architecture helps as the MCP surface expands since we give agents a better map rather than exposing all tools at the same time. Instead of exposing every task as a standalone MCP tool, we grouped related capabilities into domain-level tools with typed composable actions inside them. For example, CMS operations are grouped under a shared tool surface for collections, fields, items, publishing, and updates, while Designer capabilities are organized around elements, styles, variables, assets, and components.

More specifically, the tool surface evolved into several layers:

- **Domain tools** organize broad product areas
- **Actions** provide composable typed primitives within each domain
- **Workflow tools** encapsulate orchestration-heavy tasks
- **Guide tools** provide operational instructions and product context for safer execution

While this approach definitely helped mitigate tool surface and orchestration issues, it does not fundamentally eliminate the problem. As product capabilities expand, the number of valid workflows and actions can still grow quickly, even within a layered architecture. At some point, the challenge shifts from simply organizing tools to reducing the system's dependence on explicit tool discovery altogether. Even well-structured tool hierarchies still require the model to search, select capabilities, interpret schemas, and coordinate execution across APIs.

This is where filesystem-style abstractions and code-oriented environments become increasingly important. Instead of forcing every interaction through explicit

have both discussed similar directions in recent agent infrastructure work. See the following example for a homepage update task with the code mode:

```
// agent edits your site as code
read src/pages/home.tsx
write src/components/Hero.tsx
write src/styles/theme.css
```

This plays much more naturally to current LLM strengths. Models are already highly effective at reasoning over structured text, generating code, editing files, and operating within execution environments. By shifting the interaction model from API orchestration toward structured state manipulation, the system reduces discovery overhead, context consumption, and multi-step coordination burden.

"The deeper lesson is that scaling agent systems is not only about adding more tools. It is about giving models a more navigable and efficient operating environment."

The deeper lesson is that scaling agent systems is not only about adding more tools. It is about giving models a more navigable and efficient operating environment.

3. Observability

Observability became much more important once agents started operating autonomously across long-running workflows.

Traditional logs and traces were useful for infrastructure debugging — they could tell us which requests failed, latency patterns, or server-side exceptions. But they were poorly suited for understanding agent behavior. They did not tell us what the agent was actually trying to accomplish, which capabilities it expected to exist, where workflows became confusing, or why seemingly "successful" sessions still produced poor outcomes.

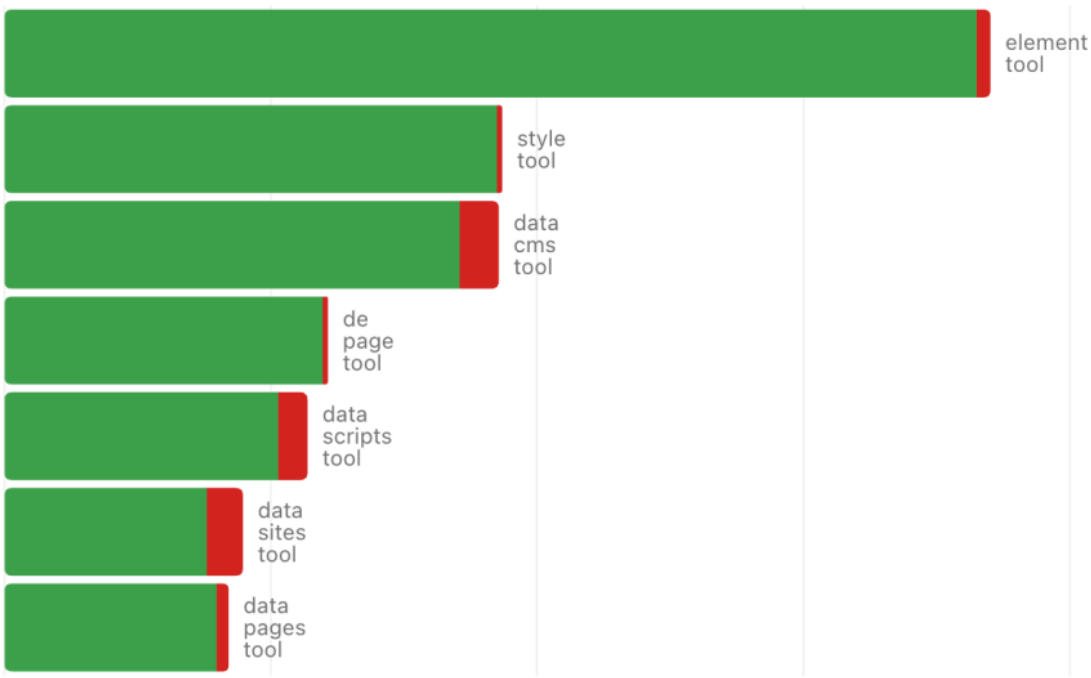
To close that gap, we integrated MCPCat as an observability layer around the MCP server itself. By wrapping the server at initialization, it captures tool calls, resource reads, missing-tool attempts, session metadata, and response patterns across the full execution flow. The most valuable signal turned out to be intent.

For the last 3 months

Canvas layout and structure	27.7%
CMS content management	16.9%
Custom code and interactions	14.1%
Project discovery	8.0%
Style and theme management	7.8%
SEO Optimization	7.0%
Responsive optimization	3.9%
CMS template management	3.7%
Localization management	2.5%
Visual styling & effects	1.9%
Other	6.4%

Most Used Tools

In the last 3 months



repeatedly explored, retried, or changed direction without ever producing a hard error. In many cases, the system appeared healthy from an infrastructure perspective while the agent experience was still failing silently.

We initially expected CMS migration and content management to dominate usage. Instead, Agent Goals showed that designing directly in the Webflow canvas through third-party agents accounted for 27.7% of sessions. We also discovered a silent error affecting more than 10% of sessions and found that 58.7% of sessions clustered around three core workflows.

Over time, we realized that observability for agent systems is fundamentally different from observability for APIs. The goal is not only understanding whether requests succeeded, but understanding how models reason, explore capabilities, recover from ambiguity, and fail during execution.

4. Scaling distribution

Scaling distribution for an MCP server was not only about handling more traffic. It was about reducing adoption friction, expanding where agents could use Webflow, and improving the quality of the workflows those agents could perform.

Adoption accelerated after we launched a hosted MCP connector in early 2026, starting with Claude, which significantly reduced setup friction for users. Instead of manually configuring MCP connections and authentication flows, users could connect to Webflow through a managed hosted integration with much less operational overhead. We've observed 6.7X session growth during the past 3 months.

Scaling adoption also made one thing clear: agents need more than tools. Tools expose what an agent can do; skills encode how to do it well. For Webflow, many successful outcomes require product judgment, reusable workflows, and domain conventions — not just API access. Bringing more useful skills into the Webflow MCP experience, such as layout patterns, CMS migration playbooks, SEO workflows, accessibility checks, and brand-system guidance, becomes critical as agents move from simple tool calls to end-to-end site work.

As adoption increased, the challenges shifted from protocol mechanics to product experience for autonomous systems: delivering richer site context, expanding the

Agents are not just another API client. They are a different kind of runtime participant: one that reasons through context, explores capabilities dynamically, learns from interface structure, and depends on the system to make the next correct action discoverable and reliable.

Designing for that kind of user changes much more than APIs. It affects how we structure tools, manage orchestration, expose context, build execution environments, observe workflows, and even how product surfaces themselves are modeled. Over time, we found ourselves designing not only MCP interfaces, but full operating environments for autonomous systems.

That pushed us toward layered tool architectures, workflow abstractions, filesystem-style project representations, richer observability, and skills layers. The goal was never simply exposing more capabilities to agents. It was helping models make correct decisions with less exploration, less orchestration burden, and greater execution reliability.

The work ahead is not just making Webflow programmable for agents. It is making Webflow legible, navigable, and operationally reliable for autonomous systems without losing the flexibility that makes the platform powerful for humans. That is both a deep product challenge and a systems challenge — and exactly the kind of problem we are excited to solve at Webflow.

This article was originally published on July 21, 2026, on webflow.com.

TNS

Yan Xie is a Director of Engineering at Webflow, where she works on agentic product initiatives. She has 12+ years of experience across applied AI, product engineering, and monetization, with a focus on building products, systems, and teams that scale...

Read more from Yan Xie →

Virat Patel is a Software Engineer at Webflow, where he leads where Webflow MCP goes next and the growing ecosystem of AI agents around it. With 12+ years in and around the web, he specializes in the connective tissue between...

Read more from Virat Patel →

developers to build, test, and ship on Webflow, and now leads teams that...

Read more from Albert Chang →

TNS owner Insight Partners is an investor in: Anthropic.

SHARE THIS STORY

TRENDING STORIES

1. Designing APIs for agents
2. The MCP debate has a context problem
3. APIs aren't dead. Here's where MCP fits alongside them.
4. 10 strategies to reduce MCP token bloat
5. Why the Model Context Protocol Won

Devops Roadmap

FOLLOW TNS